Gustafson's Law & Weak Scaling

Complete Solutions · Tasks 01–25 · Mathematical Derivations, Real Benchmarks, and Hardware Audit

9 November 2026

Abstract

A complete practice portfolio for Parallel & Distributed Computing Architecture, based on the 25-task sheet by instructor Sufyan bin Uzayr. Contains structured solutions, verified numeric calculations, a running OpenMP benchmark and independent empirical measurements. Measurements are valid only for the explicitly documented container hardware; no personal Git remote was changed.

Contents

Part I — Theoretical Foundations & Mathematical Derivations **2**
Task 01 — Strong Scaling vs. Weak Scaling Foundations . . . 2
Task 02 — Formal Derivation of Scaled Speedup . . . 3
Task 03 — The Serial Fraction Invariance Paradox . . . 4
Task 04 — Asymptotic Analysis of Parallel Efficiency . . . 5
Task 05 — Reconstructing the 1988 nCUBE/ten Result . . . 6

Part II — Mathematical Modeling & Scaled Calculations **8**
Task 06 — Parametric Speedup Across Processor Densities . . . 8
Task 07 — Maximum Tolerable Parallel Serial Overhead . . . 8
Task 08 — Cross-Over Point and Divergence: Amdahl vs. Gustafson . . . 9
Task 09 — Multi-Component Workload Partitioning . . . 10
Task 10 — Memory Growth Rate vs. Scaled Speedup . . . 11

Part III — Comparative Scaling & Theoretical Limits **13**
Task 11 — Gustafson vs. Sun–Ni Memory-Bounded Speedup . . . 13
Task 12 — Integrating Communication Overhead . . . 14
Task 13 — Concrete Cases Where Weak Scaling Breaks . . . 15
Task 14 — Isoefficiency Relation with Gustafson Scaling . . . 16
Task 15 — Energy and Power Proportionality under Scaled Speedup . . . 17

Part IV — Domain-Specific Algorithmic Workloads **19**
Task 16 — Dense Matrix Multiplication (GEMM) Weak Scaling . . . 19
Task 17 — 3-D Seven-Point Stencil and Surface-to-Volume Scaling . . . 19
Task 18 — Direct N-Body vs. Barnes–Hut Tree Weak Scaling . . . 21
Task 19 — Monte Carlo: Near-Ideal Weak Scaling with Nonzero Overhead . . . 22
Task 20 — Why BFS and PageRank Resist Regular Weak Scaling . . . 23

Part V — Benchmarking, Empirical Research & Repository Artifacts **25**
Task 21 — Parametric Speedup Simulator Implementation . . . 25
Task 22 — OpenMP Weak-Scaling Benchmark Suite . . . 25
Task 23 — Experimental Measurements and Least-Squares Fit . . . 26
Task 24 — Publication Plots and Hardware Audit . . . 27
Task 25 — Master Repository and Executive Synthesis . . . 29

Appendix — Verified Benchmark Figures **33**

Part I — Theoretical Foundations & Mathematical Derivations

Task 01 — Strong Scaling vs. Weak Scaling Foundations

Definitions and experimental question

Strong scaling (Amdahl, 1967) asks: *How much faster can a **fixed** workload finish as we add processors?* Let a reference job require one-core time T_1, of which proportion $s_A \in [0,1]$ is inherently sequential. Assuming identical processor speeds, no scheduling overhead, and ideal distribution of parallel work,

$$T_P(W_0) = T_1(W_0)\left(s_A + \frac{1-s_A}{P}\right), \qquad S_A(P) = \frac{1}{s_A + (1-s_A)/P}.$$

Here $W(P) = W_0$ **for every** P. If $s_A > 0$, then $T_P \to s_A T_1$ and $S_A(P) \to 1/s_A$ as $P \to \infty$. In a geometric problem the physical or numerical dimension N is unchanged; the subdomain assigned to each worker shrinks. Eventually, communication, minimum grain size, and launch overhead often invalidate the ideal formula even sooner.

Weak/scaled scaling (Gustafson, 1988) asks: *How much **more** work can finish in about the same time when we add processors?* Normalize the observed runtime of a scaled problem on P processors to $T_P(W(P)) = s_G + p_G = 1$; s_G and p_G represent fractions of **parallel-machine elapsed time**, not the fixed-job one-core fractions. If p_G can be multiplied by P to reconstruct sequential work, then

$$S_G(P) = \frac{s_G + P p_G}{s_G + p_G} = s_G + P p_G = P - s_G(P-1).$$

For an ideally divisible kernel, total operation count satisfies $W(P) = \Theta(P)$ and average work per core $W(P)/P = \Theta(1)$. The **dimension** scaling is determined by complexity: for a 3-D grid with $W = \Theta(N^3)$, $N(P) = \Theta(P^{1/3})$; for a dense matrix multiplication with $W = \Theta(N^3)$, the same cubic-root rule applies even though memory is $\Theta(N^2)$. It is generally **incorrect** to claim that each linear dimension must grow as P.

Limiting behavior and why the laws are not contradictory

Dimension	Amdahl/strong	Gustafson/weak
Controlled variable	Total work W_0	Approximate wall-clock budget T^*
$P \to \infty$	Same global N, smaller local work	Increasing global $N(P)$, approximately constant local work
Serial fraction	s_A on the original 1-core workload	$\alpha = s_G$ on the scaled P-core run
Question	Lower latency for one fixed result	More fidelity, samples, or throughput per time budget
Ideal speedup	Saturates at $1/s_A$ if $s_A > 0$	$P - \alpha(P-1)$ for constant α

There is **no physical contradiction**: these are different experiments, with different workloads and different denominators for the serial fraction. If W cannot expand meaningfully (e.g., a fixed-size interactive request), Amdahl's model is the appropriate planning tool. If a larger mesh, more simulations, or larger datasets produce extra scientific value and fit within capacity/time constraints, weak scaling is appropriate.

Important boundary: the word *fixed-time* describes the model's design objective, **not** a promise that production runtimes stay constant indefinitely. Communication, imbalance, memory bandwidth, and global reductions can make $T_P(W(P))$ grow.

Sources: Amdahl (1967); Gustafson (1988), *Reevaluating Amdahl's Law*; Gustafson, Montry & Benner (1988).

Task 02 — Formal Derivation of Scaled Speedup

Starting assumptions

Consider a **particular scaled job** running on $P \geq 1$ identical processors. Measure its elapsed time as a sequential component t_s and an ideally parallel component t_p in the parallel execution:

$$T_P(W(P)) = t_s + t_p, \qquad s = \frac{t_s}{T_P}, \qquad p = \frac{t_p}{T_P}, \qquad s + p = 1.$$

The parallel part contains P equal-sized units of computation executed concurrently; one processor would need Pt_p time to process all units. The sequential part still takes t_s on one processor. There is no extra serial-only cost for emulating interprocessor communication, and the one-core algorithm is assumed to perform comparable useful operations.

Step-by-step reconstruction

1. **Parallel runtime:** $T_P = t_s + t_p$ by direct phase timing.
2. **Serial-equivalent time for the *same scaled problem*:** $T_1(W(P)) = t_s + Pt_p$. The factor P applies to work that was concurrent, not to the unavoidable serial phase.
3. **Definition of speedup:**

$$S(P) = \frac{T_1(W(P))}{T_P(W(P))} = \frac{t_s + Pt_p}{t_s + t_p}.$$

4. **Normalize elapsed time:** divide numerator and denominator by $t_s + t_p$ and use $s + p = 1$:

$$S(P) = s + Pp.$$

5. **Eliminate p:** substitute $p = 1 - s$:

$$S(P) = s + P(1 - s) = P - s(P - 1).$$

6. **Rename measured serial share:** with $\alpha \equiv s = t_s/T_P$,

$$\boxed{S_G(P) = P - \alpha(P - 1)}.$$

Checkpoints: $P = 1 \implies S = 1$; $\alpha = 0 \implies S = P$; $\alpha = 1 \implies S = 1$; for $0 \leq \alpha \leq 1$ the bound $1 \leq S \leq P$ follows. Efficiency is

$$E_G(P) = \frac{S_G(P)}{P} = 1 - \alpha\left(1 - \frac{1}{P}\right).$$

Numerical consistency example

For $P = 128$ and a measured $\alpha = .08$, a normalized parallel run spends $t_s = .08$ and $t_p = .92$. A single core would spend $.08 + 128(.92) = 117.84$ units processing the **same larger job**. Therefore $S_G = 117.84/1 = 117.84$, and $E_G = .920625$. This does **not** predict a $117.84\times$ strong-scaling speedup for the original $P = 1$ problem: the job itself changed.

Conditions for the derivation

The clean expression holds only if (a) t_s and t_p partition actual elapsed time without hidden communication, (b) the parallel phase performs useful work at a processor-independent rate, (c) the serial equivalent has the assumed cost, and (d) α is measured for the processor count/workload being described. $\alpha(P)$ may change with P; writing a constant α across a sweep is an additional modeling assumption, not part of the algebraic identity.

Reference: Gustafson (1988).

Task 03 — The Serial Fraction Invariance Paradox

Compare what each fraction actually divides by

Let the serial work take $A > 0$ time on a single processor and let the ideally parallel work take $B(P)$ **one-core-equivalent** time. For the scaled job $W(P)$, ignoring communication,

$$T_1(P) = A + B(P), \qquad T_P(P) = A + \frac{B(P)}{P}.$$

The Amdahl-style fraction **for that job on one core** is

$$s_A(P) = \frac{A}{A + B(P)},$$

whereas the Gustafson fraction **measured during its P-core run** is

$$\alpha_G(P) = \frac{A}{A + B(P)/P}.$$

For the **same** workload, the two fractions satisfy

$$\boxed{\alpha_G(P) = \frac{P s_A(P)}{1 + (P-1)s_A(P)}}; \qquad \boxed{s_A(P) = \frac{\alpha_G(P)}{P - (P-1)\alpha_G(P)}}.$$

Their numeric values differ, sometimes drastically, without changing the physics or violating either speedup law.

Demonstrate the resolution

Assume weak scaling with $B(P) = Pb$ and constant $A, b > 0$. The parallel runtime is $A + b$, a constant, and hence

$$\alpha_G(P) = \frac{A}{A+b} \quad \text{(constant)}, \qquad s_A(P) = \frac{A}{A + Pb} \xrightarrow[P\to\infty]{} 0.$$

The fixed-work serial fraction is **not invariant when the workload itself grows**. In other words, the *same* A is diluted by ever more useful total work in the one-core reconstruction. The Gustafson fraction on the parallel machine can remain unchanged because each node still receives b units of parallel execution time.

For a concrete case, let $A = 1$, $b = 24$ and $P = 1024$. Then $T_P = 25$, $\alpha_G = .04$, $T_1 = 1 + 1024(24) = 24577$, and

$$S = 24577/25 = 983.08, \quad s_A(1024) = 1/24577 \approx 0.00004069.$$

Plugging this **job-specific** $s_A(P)$ into Amdahl's formula gives exactly the same 983.08, not a contradictory result. Using the fixed small-job value $s_A(1) = 1/25 = .04$ and treating it as unchanged for the expanded job would incorrectly impose an apparent ceiling of 25.

Critical note

It is therefore inaccurate to say that *measuring on the parallel machine alone removes all bottlenecks*. Constant α_G is a simplification; communication or additional coordination can make it rise. Hardware speed, cache effects, algorithmic work inflation, and one-core memory capacity also affect serial reconstruction. A 1024-node workload may be impossible to run on one physical node; its T_1 is then an **extrapolated serial equivalent** unless actually measured.

Interpretation: the paradox disappears when workload, time baseline, and fraction definition are written explicitly. Strong and weak scaling can both be valid simultaneously.

References: Gustafson et al. (1988), section on fixed-size and scaled-size evaluation.

Task 04 — Asymptotic Analysis of Parallel Efficiency

Let P be a positive integer and hold the modeled Gustafson serial fraction $0 \leq \alpha \leq 1$ constant:

$$E_G(P) = \frac{P - \alpha(P-1)}{P} = 1 - \alpha + \frac{\alpha}{P}.$$

Because $\lim_{P\to\infty} \alpha/P = 0$ for fixed finite α,

$$\boxed{\lim_{P\to\infty} E_G(P) = 1 - \alpha}.$$

For $\alpha = .04$, the asymptotic parallel efficiency is 0.96 (96%). The derivative of a real-valued relaxation, $E'_G(P) = -\alpha/P^2$, is nonpositive; efficiency decreases monotonically from $E_G(1) = 1$ to its positive asymptote. The decrement from P to $2P$ is $\alpha/(2P)$, hence falls rapidly at large P.

For a fixed **one-core** serial fraction $0 < s_A \leq 1$, Amdahl gives

$$S_A(P) = \frac{1}{s_A + (1 - s_A)/P} = \frac{P}{s_A P + 1 - s_A}.$$

Consequently,

$$E_A(P) = \frac{S_A(P)}{P} = \frac{1}{s_A P + 1 - s_A} \xrightarrow[P\to\infty]{} 0.$$

The limit follows because the denominator diverges linearly when $s_A > 0$. For the exceptional all-parallel case $s_A = 0$, $E_A(P) = 1$ at every P. Likewise, the Gustafson edge cases are $\alpha = 0 \implies E_G = 1$ and $\alpha = 1 \implies E_G = 1/P \to 0$.

Case	Scaled efficiency limit	Strong efficiency limit
Nonzero serial share, both symbols numerically .04	0.96	0
Zero serial share	1	1
Fully serial	0	0

Why the limits differ: Amdahl spreads unchanged work over more processors; Gustafson increases aggregate work with processor count while preserving a measured time composition. These do **not** concern the same sequence of jobs.

Caveat on a changing fraction: if $\alpha = \alpha(P)$ instead, then $E_G(P) = 1 - \alpha(P) + \alpha(P)/P$. A limit exists only with appropriate behavior of $\alpha(P)$ (for example, $\alpha(P) \to a$ implies $E_G(P) \to 1 - a$). No positive-efficiency limit is guaranteed in a real distributed system whose communication overhead grows with P.

Task 05 — Reconstructing the 1988 nCUBE/ten Result

Original experiment and source verification

John L. Gustafson, Gary R. Montry, and Robert E. Benner published **"Development of Parallel Methods for a 1024-Processor Hypercube"** in *SIAM Journal on Scientific and Statistical Computing* **9(4), 609–638 (July 1988)**. The work used a **1024-processor NCUBE/ten distributed-memory MIMD hypercube**, not a modern shared-memory multicore machine. The scientific applications were **wave mechanics** (explicit finite differences), **fluid dynamics** (flux-corrected transport), and **structural/beam strain analysis** (finite elements and conjugate gradients). An editorial note in the archived paper reports recognition with the Gordon Bell Award and the Karp Prize.

Crucially, the study reported **two different empirical speedup ranges**:

Experiment	Measured speedup on 1,024 processors	Interpretation
Problem size held fixed	502×–637×	Strong scaling; shrinking work per node
Problem size **per processor** held fixed	1009×–1020×	Scaled speedup; substantially larger global problems

The original paper distinguishes fixed-job serial components of roughly **0.0006–0.001** from equivalent, workload-dependent serial components of roughly **3–10 parts per million** for scaled problems. It also discusses redundant operations, operation efficiency, load imbalance, communication, and hardware-dependent floating-point rates. Its performance range is **not** evidence that all algorithms attain a 1000× gain.

Why the "0.4% serial fraction makes this impossible" statement is misleading

A **hypothetical** Amdahl calculation using a fixed-size, one-core serial share $s_A = .004$ gives a limit

$$S_A(P) \leq 1/s_A = 250.$$

At $P = 1024$, it predicts $1/[.004 + .996/1024] \approx 201.0$ for the fixed original job. This says nothing about a **different, scaled workload**, and the 0.4% example should not be falsely presented as the fixed-size fraction measured in the paper.

Under Gustafson's formulation, **if** the observed serial share of the **parallel scaled execution** were $\alpha_G = .004$, then

$$S_G(1024) = 1024 - .004(1023) = \boxed{1019.908}.$$

The actual reported range, 1009–1020, corresponds (under the ideal two-component model) to effective α_G values

$$\frac{1024 - 1020}{1023} \approx 0.00391 \quad \text{through} \quad \frac{1024 - 1009}{1023} \approx 0.01466.$$

These are *parallel-runtime* fractions; their values must not be swapped with the Amdahl fraction of a different job. The reported efficiencies for scaled runs were approximately 98.5%–99.6%; high efficiency was achieved by keeping much larger useful work per node and controlling communication.

Architecture and practical mechanisms

The NCUBE/ten nodes communicated through explicit point-to-point links; each node had local memory. The scientists designed larger spatial domains, high-granularity updates, pipelined or localized messaging, and selective redundant computations to reduce communication stalls. The paper explicitly notes that for very large scaled domains a literal one-node serial run might be infeasible in memory;

in such cases serial-equivalent speedups are **extrapolations** based on operation efficiency and timing, not stopwatch measurements of an impossible run.

Conclusion

The breakthrough did **not** falsify Amdahl's mathematical bound. It exposed the error of silently extending **one small-job serial fraction** to a family of expanded scientific jobs. The general lesson is to report *both* strong and scaled speedups with workload size, core count, and serial-equivalence methodology.

Primary references: Original SIAM paper; archived full version; Gustafson, "Reevaluating Amdahl's Law," CACM 31(5), 1988.

Part II — Mathematical Modeling & Scaled Calculations

Task 06 — Parametric Speedup Across Processor Densities

Given: the observed serial fraction on the parallel machine is $\alpha = 0.04$; $P \in \{16, 64, 256, 1024, 4096, 16384\}$.

$$S_G(P) = P - 0.04(P-1) = 0.96P + 0.04,$$
$$E_G(P) = \frac{S_G(P)}{P} = 0.96 + \frac{0.04}{P}.$$

Complete computed table

Processors P	Serial penalty $0.04(P-1)$	Scaled speedup S_G	Efficiency E_G	Efficiency (%)
16	0.60	15.40	0.9625000000	96.250000
64	2.52	61.48	0.9606250000	96.062500
256	10.20	245.80	0.9601562500	96.015625
1,024	40.92	983.08	0.9600390625	96.003906
4,096	163.80	3,932.20	0.9600097656	96.000977
16,384	655.32	15,728.68	0.9600024414	96.000244

Calculation log

- $P = 16:\ S = 16 - .04(15) = 15.40;\ E = 15.40/16 = .9625.$
- $P = 64:\ S = 64 - .04(63) = 61.48;\ E = 61.48/64 = .960625.$
- $P = 256:\ S = 256 - .04(255) = 245.80;\ E = 245.80/256 = .96015625.$
- $P = 1024:\ S = 1024 - .04(1023) = 983.08;\ E = 983.08/1024 = .9600390625.$
- $P = 4096:\ S = 4096 - .04(4095) = 3932.20;\ E = 3932.20/4096 = .960009765625.$
- $P = 16384:\ S = 16384 - .04(16383) = 15728.68;\ E = 15728.68/16384 = .96000244140625.$

Interpretation

Efficiency declines by **0.249756 percentage points** from 96.250000% at 16 processors to roughly 96.000244% at 16,384, approaching but never falling below its fixed-α asymptote of 96%. The *absolute* lost ideal speedup, $P - S_G(P) = .04(P-1)$, still grows linearly: missing 655.32 units of ideal speedup at 16,384 processors. This is a mathematical model, not a prediction that an actual 16k-core cluster maintains constant 4% nonparallel elapsed time.

Independent verification: `benchmarks/simulator.py --core-counts 16,64,256,1024,4096,16384 --alphas .04` generates the same figures in a CSV.

Task 07 — Maximum Tolerable Parallel Serial Overhead

Given $P = 8192$ processing cores and an operational minimum scaled speedup $S_G(P) \geq 7500$, solve for the maximum fraction α of the **parallel machine's elapsed time** that can be serial.

$$S_G(8192) = 8192 - \alpha(8192 - 1) = 8192 - 8191\alpha.$$

Apply the speedup requirement:

$$8192 - 8191\alpha \geq 7500$$
$$-8191\alpha \geq -692$$
$$\boxed{\alpha \leq \frac{692}{8191} \approx 0.0844829691124}.$$

Therefore the maximum is $\alpha_{\max} \approx 8.44829691124\%$ **of parallel runtime**.

Verification

Substituting the *exact* rational limit gives $S_G = 8192 - 8191(692/8191) = 8192 - 692 = 7500$, which satisfies the boundary at equality. For any $\alpha > 692/8191$, the predicted speedup is strictly less than 7500. For $\alpha < 692/8191$, it exceeds the requirement.

The implied minimum scaled efficiency is

$$E_{\min} = \frac{7500}{8192} = \frac{1875}{2048} \approx 0.91552734375 = 91.552734375\%.$$

Practical reading

The allowed fraction includes **all nonparallel elapsed-time overhead classified as serial in the simplified Gustafson model**; it should not be confused with a fixed-job, single-core Amdahl fraction. If communication has a separate non-overlapped time fraction γ, the extended condition becomes

$$8192 - 8191\alpha - 8192\gamma \geq 7500 \iff 8191\alpha + 8192\gamma \leq 692.$$

A real operational threshold should also account for trial-to-trial variance, outliers, thermal effects, and service-level safety margin.

Task 08 — Cross-Over Point and Divergence: Amdahl vs. Gustafson

Given: processor count $P = 128$ and the **same numeric value** 0.08 used as two conceptually different fractions: $s_A = .08$ of a one-core fixed workload and $\alpha_G = .08$ of a 128-core scaled workload.

Amdahl (fixed original job)

$$S_A(128) = \frac{1}{0.08 + 0.92/128}$$
$$= \frac{1}{0.08 + 0.0071875} = \frac{1}{0.0871875}$$
$$\boxed{S_A(128) = 11.4695340502\times}.$$

The maximum possible fixed-job speedup for this s_A is $1/.08 = 12.5$ as $P \to \infty$.

Gustafson (larger job, constant time)

$$S_G(128) = 128 - 0.08(128 - 1) = 128 - 10.16$$
$$\boxed{S_G(128) = 117.84\times}, \qquad E_G = 117.84/128 = 0.920625.$$

Numerical discrepancy (explicit denominator)

Absolute difference:

$$\Delta S = S_G - S_A = 117.84 - 11.4695340502 = \boxed{106.3704659498}.$$

Percentage **relative to Amdahl** (chosen primary convention):

$$\frac{S_G - S_A}{S_A} \cdot 100\% = \boxed{927.4175\%}.$$

Equivalently $S_G/S_A = 10.274175$. If one instead uses Gustafson as the denominator, the discrepancy is about **90.2668584%**. Always state the denominator because "percentage difference" is otherwise ambiguous.

Are these predictions contradictory?

No. The numbers use **equal labels but unequal measurement baselines**. s_A refers to the one-core *fixed* job; α_G refers to runtime fractions during a *scaled* 128-core run. A valid same-workload comparison must transform the fractions. If $\alpha_G = .08$ on $P = 128$, its equivalent Amdahl one-core fraction for the *scaled* job is

$$s_{A,\text{scaled}} = \frac{\alpha_G}{P - (P-1)\alpha_G} = \frac{0.08}{117.84} \approx 0.000678887.$$

Inserting this transformed share in Amdahl gives exactly 117.84. Hence the task's "cross-over" comparison is a **cross-model numerical comparison**, not a physical intersection or contradiction. The apparent gap grows as the core count increases because one formula fixes useful work while the other expands it.

Task 09 — Multi-Component Workload Partitioning

The parallel execution of a scientific simulation consists of:

Phase	Activity	Share of observed parallel elapsed time	Model treatment
A	Disk I/O + parameter broadcast	2%	serial/nonoverlapped
B	Boundary exchange synchronization	6%	serial/nonoverlapped (conservative assumption)
C	Core-domain flux computation	92%	ideally parallelizable
Total		**100%**	

Under the exercise's simplified classification, the **aggregate serial fraction** on the parallel machine is

$$\boxed{\alpha = 0.02 + 0.06 = 0.08}, \qquad p = 0.92.$$

The scaled-speedup prediction is

$$S_G(P) = P - 0.08(P-1) = 0.08 + 0.92P.$$

For **512 processors**,

$$S_G(512) = 0.08 + 0.92(512) = \boxed{471.12},$$
$$E(512) = 471.12/512 = \boxed{0.92015625}\,(92.015625\%).$$

For **2,048 processors**,

$$S_G(2048) = 0.08 + 0.92(2048) = \boxed{1884.24},$$
$$E(2048) = 1884.24/2048 = \boxed{0.9200390625}\,(92.00390625\%).$$

Sensitivity and architectural interpretation

The 6% boundary exchange is **not inherently serial computation** in all implementations. If asynchronous halos hide a proportion h of that 6%, then the non-overlapped overhead is $\alpha_{\text{effective}} = 0.02 + 0.06(1-h)$. For example, overlap of half that cost gives $\alpha = .05$ and $S_G(512) = 512 - .05(511) = 486.45$. This is an alternative hypothetical, not the result dictated by the task.

Conversely, if the boundary cost rises with processor count, the constant 6% premise fails and the measured $\alpha(P)$ must be updated. A more accurate instrumentation plan separates disk time, serialized control work, wait time, network transit, and useful floating-point execution; communication may be tracked independently using Task 12's $\gamma(P)$ term.

Task 10 — Memory Growth Rate vs. Scaled Speedup

General capacity constraint

Under pure weak scaling, $W(P) = PW(1)$ and total memory $M(P) = cP$. Assume P **nodes**, each with a hard cap of **32 GiB** of accessible memory (1 GiB $= 2^{30}$ bytes). For a cubic 3-D grid with N cells along each side and b bytes of *total persistent memory per grid cell*, the leading storage requirement is

$$M_{\text{grid}}(N) = bN^3 \text{ bytes.}$$

The aggregate cap is $P \cdot 32 \cdot 2^{30}$ bytes. Therefore,

$$bN^3 \le 32P \cdot 2^{30}, \quad \boxed{N_{\max}(P,b) = \left\lfloor \left(\frac{32P \cdot 2^{30}}{b} \right)^{1/3} \right\rfloor}.$$

Important: the assignment does **not** specify b, nor halo buffers or runtime allocations. Consequently it does **not determine a unique numerical** N. To provide a concrete answer, the table explicitly assumes **one 64-bit (8-byte) scalar per cell**, zero halos, perfectly distributable storage, and that *all* 32 GiB on every node can be used. This is an optimistic **capacity ceiling**, not a production-safe grid size.

Worked capacity ceilings, with stated assumptions

Nodes P	Aggregate memory	Assumed bytes/cell b	Maximum integer global side N	Total cells N^3 (approx.)
64	2 TiB	8	**6,501**	2.747×10^{11}
512	16 TiB	8	**13,003**	2.198×10^{12}
64	2 TiB	16	**5,160**	1.374×10^{11}
512	16 TiB	16	**10,321**	1.099×10^{12}

For $b = 8$ and $P = 64$:

$$N \le \left(\frac{64 \cdot 32 \cdot 2^{30}}{8} \right)^{1/3} \approx 6501.995 \implies N_{\max} = 6501.$$

For $b = 8$ and $P = 512$:

$$N \le \left(\frac{512 \cdot 32 \cdot 2^{30}}{8} \right)^{1/3} \approx 13003.989 \implies N_{\max} = 13003.$$

The eightfold increase in node count raises the ideal cubic grid **side** by a factor of $8^{1/3} = 2$, while increasing total cell count by roughly 8.

Real grid allocation caveats

The formula assumes that the distributed grid maps perfectly onto nodes. A real solver needs multiple state arrays, halos, pressure/velocity fields, scratch storage, MPI buffers, checkpointing, and runtime reserves. If the usable fraction of node memory is $\eta_m < 1$, replace 32 GiB with $32\eta_m$ GiB. Non-cubic decompositions may waste memory, and partitions should have convenient integer dimensions. If "32 GB" is defined in decimal rather than GiB, replace 2^{30} with 10^9; the numerical ceilings decrease.

Scaling conclusion: total capacity $\Theta(P)$ is consistent with $N = \Theta(P^{1/3})$; however storage feasibility alone does not prove constant elapsed time or preserved Gustafson efficiency.

Part III — Comparative Scaling & Theoretical Limits

Task 11 — Gustafson vs. Sun–Ni Memory-Bounded Speedup

Three different resource constraints

1. **Amdahl fixed-size:** $W(P) = W_0$; available processors increase but required result does not.
2. **Gustafson fixed-time:** choose $W(P)$ so $T_P(W(P)) \approx T^*$; in the ideal two-phase model, $S_G(P) = P - \alpha(P-1)$.
3. **Sun–Ni memory-bounded:** choose the largest useful problem whose data fit into aggregated memory $M(P)$, even if runtime changes with P. With m bytes accessible per node, $M(P) \leq Pm$ and $W^*(P) = g(M(P))$, where g maps memory footprint to work.

Sun and Ni's 1993 work explicitly treats memory capacity as another scalability constraint and includes the fixed-size and fixed-time models as special cases under simplified assumptions.

Mathematical formulation

Let A denote constant sequential work (in time units) and B the parallelizable work at the baseline memory footprint m. Write the scaled parallel work as $B(P) = h(P)B$, where for a semihomogeneous memory–work map g one may take

$$h(P) = \frac{g(Pm)}{g(m)}.$$

With perfect distribution and no communication, memory-bounded speedup on the **larger job** is

$$\boxed{S_M(P) = \frac{A + h(P)B}{A + h(P)B/P}}.$$

Let $s_0 = A/(A+B)$, the baseline one-core fraction. Dividing by $A + B$ gives

$$S_M(P) = \frac{s_0 + h(P)(1-s_0)}{s_0 + h(P)(1-s_0)/P}.$$

- If $h(P) = 1$, this reduces to **Amdahl** for the fixed workload.
- If $h(P) = P$, the denominator is $s_0 + 1 - s_0 = 1$ and $S_M(P) = s_0 + P(1-s_0)$: the **Gustafson** fixed-time form with $\alpha = s_0$ for this particular scaled schedule.
- If $g(M) \propto M^b$, then $h(P) = P^b$. For a dense $N \times N$ product, $M \propto N^2$ and work $\propto N^3$, so $b = 3/2$; using all aggregate memory could make useful work grow as $P^{3/2}$ and compute time as $P^{1/2}$ rather than remain fixed.

A realistic generalization includes communication and imbalance:

$$S_M^{\text{real}}(P) = \frac{A + h(P)B}{A + h(P)B/P + C(P) + I(P)}.$$

Here $C(P)$ and $I(P)$ are *time* costs, not improvements in useful work. Memory capacity is only an upper bound: inadequate network bandwidth or inaccessible remote memory can reduce usable $M(P)$.

When Gustafson is too optimistic or too conservative

Overestimates achievable throughput when (i) all memory cannot be accessed efficiently, (ii) communication grows with domain size, (iii) aggregate memory capacity is exhausted before constant-runtime work grows to P times baseline, (iv) dependencies or load imbalance increase, or (v) the best serial algorithm does fewer operations than the parallel implementation. Example: if $h(P) = P^{1/2}$

under a memory restriction and $A > 0$, the serialized portion has a greater relative effect than an assumed $h(P) = P$ would suggest.

Underestimates useful work that can be performed if the time budget may expand and memory enables superlinear growth of operations relative to memory ($h(P) > P$): dense linear algebra can perform $\Theta(P^{3/2})$ arithmetic when matrix memory scales as P. This does **not** imply superlinear *speedup of the same algorithm in the same time*; the memory-bounded run may take longer. Another apparent underestimate arises if larger working sets improve throughput or the serial component shrinks, violating constant-α assumptions.

Key distinction: a *speedup ratio*, a *quantity of useful work*, and *throughput per fixed time* are different outcomes. Models are not directly rankable without declaring which constraint and numerator are held fixed.

Primary reference: X.-H. Sun and L. Ni, "Scalable Problems and Memory-Bounded Speedup," *Journal of Parallel and Distributed Computing* 19(1), 27–37 (1993). NASA preprint record.

Task 12 — Integrating Communication Overhead

A time-domain extension

Gustafson's model assumes that all nonserial elapsed time corresponds to useful, perfectly distributable computation. That becomes misleading when workers also spend time communicating. For a scaled problem let

- $A(P)$ = useful sequential computation time,
- $B(P)$ = one-core-equivalent useful parallel computation time,
- $C(P)$ = elapsed, non-overlapped communication and synchronization time on P processors.

Then an explicit extended runtime model is

$$T_P(P) = A(P) + \frac{B(P)}{P} + C(P).$$

Assume that the sequential reference performs the same useful computation but does **not** pay the multi-node network cost: $T_1(P) = A(P) + B(P)$. Measured scaled speedup becomes

$$\boxed{S_{G,C}(P) = \frac{A(P) + B(P)}{A(P) + B(P)/P + C(P)}}.$$

This form is often preferable because the time budget need not be fixed and the communication function need not be small.

Recovery of Gustafson and a modified closed form

Normalize $T_P(P) = 1$ and define $\alpha(P) = A(P)/T_P(P)$, $\gamma(P) = C(P)/T_P(P)$; therefore $B(P)/[PT_P(P)] = 1 - \alpha(P) - \gamma(P)$. Substituting yields

$$S_{G,C}(P) = \alpha(P) + P[1 - \alpha(P) - \gamma(P)]$$

$$\boxed{S_{G,C}(P) = P - \alpha(P)(P-1) - P\gamma(P).}$$

If $C(P) = 0$, the original expression is recovered. The penalty term $P\gamma(P)$ represents lost useful-work-equivalent speedup. **Do not** simply add $C(P)$ to a fraction already normalized as $s + p = 1$ without renormalizing.

Network cost examples

For a tree-based global collective with k synchronization rounds,

$$C_{\text{tree}}(P) = k\lambda \log_2 P + \frac{V(P)}{B_{\text{net}}(P)},$$

where λ is message-latency per tree stage and V/B_{net} a nonoverlapped effective bandwidth time. On a 2-D mesh, a diameter-sensitive exchange may have

$$C_{\text{mesh}}(P) = c\sqrt{P} + \text{bandwidth term}.$$

The Big-O forms $O(\log P)$ and $O(P^{1/2})$ are examples of **network/algorithm combinations**, not universal laws for all communication on those topologies. Constant communication volume per worker may produce different costs, and overlapping data transfer with computation reduces the *nonoverlapped* $C(P)$.

Fixed-time feasibility bound

With target $T^* = 1$ and a fixed sequential component $A = a$, useful work that fits the budget is

$$B(P) = P[1 - a - C(P)].$$

The implied useful workload is nonnegative only when $a + C(P) \leq 1$. If $C(P) = c \log_2 P$ with $c > 0$, this fails beyond $P > 2^{(1-a)/c}$. Thus **constant wall-clock time cannot continue indefinitely** under an unbounded communication penalty.

Efficiency becomes $E(P) = S(P)/P = 1 - \alpha(P) + \alpha(P)/P - \gamma(P)$. This shows why it is possible to observe improving worker utilization but decreasing effective speedup when collective latency, contention, or redundant work increases.

Use: fit A, B, and C separately from timed traces; avoid assigning all communication to inherently serial algorithms when parallel overlap is available.

Task 13 — Concrete Cases Where Weak Scaling Breaks

Case A — 3-D FFT with distributed transposes

A pencil-decomposed three-dimensional FFT can scale floating-point work by increasing the global grid. Suppose each worker owns a constant number k of points, so total points $N_{\text{tot}} = kP$. Local arithmetic is approximately $\Theta(k \log(kP))$ per worker, **not** a constant. Global FFT stages typically require all-to-all redistribution (or several transposes) across subsets of the process grid.

A conceptual per-iteration runtime is

$$T_P = a\, k \log(kP) + T_{\text{alltoall}}(P, k) + T_{\text{plan}}(P).$$

Even if a simplified all-to-all delivers a constant payload per worker, many small messages or increasing contention can raise latency, for example $T_{\text{alltoall}} \gtrsim \lambda\sqrt{P}$ for some pencil communication patterns. More advanced all-to-all implementations may have better constants, but a **growing** $\log P$ compute term already violates a strict fixed-time claim at constant local grid size.

Failure mechanism: the algorithmic complexity per stored grid point depends on the global dimension, and global data redistribution requires nonlocal synchronization. Adding nodes does not preserve a constant amount of work *or* communication time per node.

Mitigations: larger batch sizes, overlapping transposes, topology-aware process layouts, fewer transform stages, mixed decomposition strategies. Mitigations can shift the break point but cannot make a universal constant-time proof.

Case B — Breadth-first search on a scale-free graph

For BFS, expanding a graph with $V(P) = vP$ vertices and average degree near constant implies $E(P) = \Theta(P)$ total edges, seemingly an ideal constant local edge count. But the work per BFS level depends on the *frontier*, not merely total edge count. A few high-degree hubs in a power-law graph cause uneven partition loads; each frontier may require remote reads, atomics, and global decisions that cannot be kept fully local.

One coarse model is

$$T_P = \sum_{\ell=1}^{D(P)} \left(\frac{E_\ell(P)}{Pr} + C_\ell(P) + I_\ell(P) \right),$$

where $D(P)$ is the number of BFS levels, E_ℓ edges visited at a level, r the worker edge rate, C_ℓ communication/synchronization, and I_ℓ imbalance. Neither $D(P)$ nor the maximum frontier work per worker is guaranteed constant. For example, if one hub's degree grows as P^β and its adjacency remains a single-owner partition, the maximum work on that owner grows as $\Omega(P^\beta)$ even though *mean* edges per worker remain constant.

Failure mechanism: skewed partitions and global frontier coordination dominate the nominal constant local volume. Average weak scaling hides tail latency from the most heavily loaded partition.

Mitigations: direction-optimizing BFS, 2-D partitioning, work stealing, hub replication, batching updates. Their own communication and replication costs must be measured.

General lesson

The condition $W(P) = \Theta(P)$ is **necessary for ideal weak scaling of a constant-work-per-worker kernel, but not sufficient**. A compliant algorithm also needs bounded critical-path length, bounded imbalanced local work, bounded nonoverlapped communication, and bounded memory access costs. $\alpha(P)$ and $C(P)$ must be measured rather than inferred from the operation count alone.

Task 14 — Isoefficiency Relation with Gustafson Scaling

Formal definitions

Let useful serial computation on a workload of size W take $T_1(W) = W$ time units, and the parallel execution take $T_P(W)$. The total parallel overhead in **work units** is

$$T_o(P, W) = PT_P(W) - T_1(W).$$

Speedup and efficiency are

$$S(P, W) = \frac{W}{T_P(W)}, \qquad E(P, W) = \frac{S}{P} = \frac{W}{W + T_o(P, W)}.$$

To maintain a prescribed efficiency $E_0 \in (0, 1)$,

$$E(P, W) \geq E_0 \iff W \geq \frac{E_0}{1 - E_0} T_o(P, W).$$

The **isoefficiency function** is the growth rate of $W(P)$ necessary to keep this inequality satisfied as P increases. Since T_o can depend on W, the condition may have to be solved implicitly.

Proof for a Gustafson-compliant constant-serial-time model

Take $A = a > 0$ time of serial work and $B(P) = bP$ units of perfectly distributable work ($b > 0$). Then

$$W(P) = a+bP, \qquad T_P(W(P)) = a+b, \qquad T_o(P, W(P)) = P(a+b)-(a+bP) = (P-1)a.$$

Hence $T_o = \Theta(P)$, and the equation for fixed positive efficiency requires $W(P) = \Omega(P)$; a linear workload $a + bP$ is sufficient. Indeed,

$$E(P) = \frac{a+bP}{P(a+b)} = \frac{b}{a+b} + \frac{a}{P(a+b)} \longrightarrow \frac{b}{a+b} = 1-\alpha.$$

Thus, for this **particular** overhead model, the minimum isoefficiency is $W(P) = \Theta(P)$, which matches standard weak scaling. The claim is conditional, not a theorem that all programs with a `Gustafson` label have linear isoefficiency.

What communication changes

Suppose nonoverlapped communication costs $C(P) = c \log P$ per fixed-size local partition. Then at $W = bP$,

$$T_o(P, W) = (P-1)a + Pc \log P = \Theta(P \log P).$$

But $W(P) = \Theta(P)$ fails $W \geq KT_o$ as $P \to \infty$, so efficiency declines. Under a workload model where increasing local computation is permitted and communication remains $O(\log P)$ independent of local size, one would need $W(P) = \Omega(P \log P)$ to hide this cost at fixed efficiency, breaking *strict* constant local-work weak scaling. With additional per-message data movement the isoefficiency may be even worse.

Takeaway: Gustafson's simple expression implicitly assumes overhead grows no faster than total useful work, and that serial duration remains bounded when growing the job. Isoefficiency exposes the extra assumption: constant work **per processor** is viable only when the overhead-to-work ratio is controlled.

Reference: classic isoefficiency analysis as described by parallel-algorithm texts by Kumar, Grama, Gupta, and Karypis; derivation above follows directly from $E = W/(W + T_o)$.

Task 15 — Energy and Power Proportionality under Scaled Speedup

Distinguish power (watts) from processor count P

Let each **powered-on** node draw idle/static power p_{idle} watts, plus dynamic compute power p_{active} watts while active. Let a weak-scaled run finish in constant wall-clock time T with serial elapsed fraction α, and assume only one node performs useful compute during the serial phase while the other $P-1$ nodes remain powered on and idle. All P compute during the parallel phase $1-\alpha$.

The idle/static draw is always Pp_{idle}. Dynamic power is p_{active} for αT, and Pp_{active} for $(1-\alpha)T$. Integrating power over elapsed time gives

$$\boxed{\mathcal{E}_P = T\left[Pp_{\text{idle}} + (\alpha + P(1-\alpha))\, p_{\text{active}}\right]}.$$

The energy for a **single core performing its own smaller one-core weak-scaling workload** in the same time T is

$$\mathcal{E}_{1,\text{small}} = T(p_{\text{idle}} + p_{\text{active}}).$$

Their ratio, which compares **different amounts of work**, is

$$\frac{\mathcal{E}_P}{\mathcal{E}_{1,\text{small}}} = \frac{Pp_{\text{idle}} + [\alpha + P(1-\alpha)]p_{\text{active}}}{p_{\text{idle}} + p_{\text{active}}}.$$

This ratio normally grows approximately linearly with P, because the larger system performs much more useful work. It is **not** proof of energy inefficiency.

Apples-to-apples comparison: same scaled job

Under the ideal model, the same scaled job would take $T_1(W(P)) = S_G(P)T$ on one core, with

$$S_G(P) = \alpha + P(1-\alpha).$$

Single-core energy for that large job is

$$\mathcal{E}_{1,\text{same}} = S_G(P)T(p_{\text{idle}} + p_{\text{active}}).$$

Hence the energy ratio for equal work is

$$\boxed{\frac{\mathcal{E}_P}{\mathcal{E}_{1,\text{same}}} = \frac{Pp_{\text{idle}} + S_G(P)p_{\text{active}}}{S_G(P)(p_{\text{idle}} + p_{\text{active}})}.}$$

In this simple model dynamic energy for useful compute is **the same** on one core and P cores ($S_G T p_{\text{active}}$). The difference comes from powering idle baseline electronics on extra nodes. For $\alpha > 0$, some static energy may be wasted while nodes wait at the serial phase.

Numerical illustration

Take $P = 128$, $\alpha = .04$, $T = 100\,s$, $p_{\text{idle}} = 40\,W$, $p_{\text{active}} = 60\,W$. Then $S_G = 122.92$, and

- Parallel run: $\mathcal{E}_P = 100[128(40) + 122.92(60)] = 1{,}249{,}520\,J$.
- One core running its **small** job for 100 s: $\mathcal{E}_{1,\text{small}} = 10{,}000\,J$ (but processes much less work).
- One core running the **same large** job for $12{,}292\,s$: $\mathcal{E}_{1,\text{same}} = 1{,}229{,}200\,J$.

This simplified example gives about 1.0165× the energy for the parallel machine versus one core on identical work, while finishing 122.92× sooner. Real power curves depend on DVFS, leakage, DRAM, network, cooling, utilization, and node power-down policy. **Energy per solved timestep/sample and joules per useful result** should accompany scaled speedup.

Part IV — Domain-Specific Algorithmic Workloads

Task 16 — Dense Matrix Multiplication (GEMM) Weak Scaling

Work-derived matrix dimension

For conventional dense multiplication of square $N \times N$ matrices, useful arithmetic is $W(N) = \Theta(N^3)$ while input/output storage is $M(N) = \Theta(N^2)$. Write the leading work $W(N) = cN^3$. Under **ideal constant-time weak scaling** on P identical processors, arithmetic handled by each core stays constant:

$$\frac{cN(P)^3}{P} = cN_0^3 \quad \Longrightarrow \quad N(P)^3 = PN_0^3.$$

Thus,

$$\boxed{N(P) = N_0 P^{1/3}}.$$

The fixed-time computation assumption leads to a **cube-root** increase of matrix side length, *not* $N \propto P$. The result is approximate for integer matrix sizes and assumes that FLOP rate per processor, local arithmetic efficiency, and communication can remain stable.

Tabular dimension growth targets

For an illustrative one-core baseline $N_0 = 256$:

Processors	Multiplicative side factor $P^{1/3}$	Target $N(P)$	Aggregate FLOPs relative to baseline	Matrix storage relative to baseline
1	1	256	1×	1×
8	2	512	8×	4×
64	4	1,024	64×	16×
512	8	2,048	512×	64×
4,096	16	4,096	4,096×	256×
32,768	32	8,192	32,768×	1,024×

Since $M(N(P)) = \Theta(P^{2/3})$, the ideal **storage per processor** is $M/P = \Theta(P^{-1/3})$. In this regime more processors are added for compute throughput, not to saturate their aggregate available memory. A memory-filling experiment would instead allow $M \propto P$, yielding $N \propto P^{1/2}$ and total GEMM work $\propto P^{3/2}$, so naive compute time $W/P \propto P^{1/2}$: that is Sun–Ni **memory-bounded** scaling, not fixed-time Gustafson scaling.

Communication qualification

Distributed GEMM uses broadcasts/reductions of matrix blocks and may replicate panels for efficiency. An ideal 2-D process grid's bandwidth cost per processor has a nontrivial dependence on N and P; a formula based only on N^3/P omits it. For a real strong-scaling or weak-scaling claim, benchmark total wall time including block redistribution, initialization, and final reduction/checkpointing; report grid arrangement, BLAS library, and per-core memory.

Task 17 — 3-D Seven-Point Stencil and Surface-to-Volume Scaling

Global and local decomposition

Take a global cubic grid of linear dimension N (so N^3 update points) split over $P = q^3$ processors in a regular $q \times q \times q$ arrangement. Each owns a local cube of side

$$n = \frac{N}{q} = \frac{N}{P^{1/3}}.$$

Interior work per iteration per processor is approximately $c_w n^3$; for a 7-point finite-difference stencil, each interior update accesses six neighbors and the center. Exchanging one-cell-wide halos with six neighboring partitions transfers approximately $6n^2$ values per processor, excluding external domain boundaries. With b bytes per value,

$$\text{compute/node} = \Theta(n^3), \quad \text{halo bytes/node} \approx 6bn^2.$$

Hence the halo-volume-to-computation ratio for one iteration obeys

$$\boxed{R_{\text{comm/comp}} \propto \frac{6bn^2}{c_w n^3} = \frac{6b}{c_w n}}.$$

What happens under true weak scaling?

For fixed wall-time and constant local work, scale the **global number of cells** proportionally to processor count:

$$N(P)^3 = PN_0^3 \quad \Rightarrow \quad N(P) = N_0 P^{1/3}.$$

Then $n = N(P)/P^{1/3} = N_0$ stays **constant**, giving

$$\boxed{R_{\text{comm/comp}} = \Theta(1).}$$

Therefore the **per-processor** communication/computation ratio does **not** improve merely because the global domain expands. Each worker still computes on N_0^3 cells and exchanges $\sim 6N_0^2$ halo elements. In an ideal nearest-neighbor network with constant latency and bandwidth, constant local runtime is plausible.

Why a global surface-to-volume argument can mislead

The *outside surface* of the entire global cube is $\Theta(N^2)$ while its volume is $\Theta(N^3)$, so the global external ratio is $\Theta(1/N) = \Theta(P^{-1/3})$. But most interprocessor traffic comes from **internal partition interfaces**, not the outside domain boundary. Summing halo exchanges over P local cubes yields $\Theta(Pn^2)$ words and useful updates $\Theta(Pn^3)$; their ratio remains $\Theta(1/n)$.

If instead local cubes become larger ($n \uparrow$) at a fixed processor count, the ratio **improves** as $1/n$. If strong scaling fixes global N and increases P, then $n = N/P^{1/3} \downarrow$ and the ratio **degrades** as $P^{1/3}/N$.

Latency and topology

A realistic step may cost

$$T_{\text{step}} \approx c_w n^3 + 6\lambda + \frac{6bn^2}{B_{\text{eff}}(P)} + T_{\text{sync}}(P).$$

Even under weak scaling, a declining effective network bandwidth or growing global synchronization cost makes T_{step} increase. The result is therefore **conditional weak-scaling compliance**, not a guarantee of constant time on every cluster.

Task 18 — Direct N-Body vs. Barnes–Hut Tree Weak Scaling

Two different work laws

Direct all-pairs interactions: With N particles, the pairwise gravitational force evaluation requires $W_D(N) = \Theta(N^2)$ interactions. Ideal P-processor per-iteration compute time is proportional to N^2/P. To keep that compute time constant as processor count rises,

$$N_D(P)^2/P = N_0^2 \quad \Longrightarrow \quad \boxed{N_D(P) = N_0\sqrt{P}}.$$

Barnes–Hut tree code: Under conventional balanced-tree assumptions and a fixed acceptance criterion, expected computational work is approximately $W_{BH}(N) = \Theta(N \log N)$. To preserve constant per-worker compute time,

$$\frac{N_{BH}(P) \ln N_{BH}(P)}{P} = N_0 \ln N_0.$$

Let $A = N_0 \ln N_0 > 0$. Setting $N \ln N = AP$ and using the Lambert W function ($W(z)e^{W(z)} = z$),

$$\boxed{N_{BH}(P) = \frac{AP}{W(AP)}}.$$

Asymptotically $N_{BH}(P) = \Theta(P/\ln P)$ up to lower-order log terms. The tree method permits faster particle-count growth with P under the **same constant compute-time target** because the serial algorithm has a lower exponent.

What happens to serial-to-parallel time ratios?

Neither $O(N^2)$ nor $O(N \log N)$ alone specifies the **serial overhead**. Let $A_{\text{serial}}(P)$ be unoverlapped setup, tree build, communication, scheduling, or reduction time, and $B_{\text{local}}(P)$ be parallel compute time per processor. The observed ratio is

$$r(P) = \frac{A_{\text{serial}}(P)}{B_{\text{local}}(P)}.$$

By construction, both methods target $B_{\text{local}}(P) = \Theta(1)$. Therefore **any unbounded serial/control overhead makes $r(P)$ grow**, contradicting a constant-time premise. Specific examples illustrate the risk:

Regime	Hypothesized non-overlapped overhead	Scaling at fixed compute per worker
Direct, only constant control setup	$\Theta(1)$	$r_D = \Theta(1)$: plausible ideal weak scaling
Direct, centralized $\Theta(N)$ particle distribution	$\Theta(N)$	$r_D = \Theta(\sqrt{P})$: worsens
Barnes–Hut, centralized $\Theta(N)$ tree construction	$\Theta(N)$	$r_{BH} = \Theta(P/\log P)$: severe growth
Either method, only logarithmic global reduction	$\Theta(\log P)$	$r = \Theta(\log P)$: slow but unbounded growth

These are **conditional illustrations**, not claims that a well-designed Barnes–Hut code necessarily builds its tree serially. Parallel tree construction, domain decomposition, hierarchical communication, and data redistribution can reduce these overheads. Direct methods may be communication-bound if particle positions must be replicated or exchanged globally each step. Barnes–Hut may incur remote multipole lookups, load imbalance in clustered domains, and tree-rebuilding costs.

Algorithmic comparison and limitations

For equal *particle count* N, Barnes–Hut reduces arithmetic significantly but introduces approximate force error, tree storage, branching irregularity, and distributed data-dependence. For equal fixed-time target under increasing P, direct summation can only raise particle count like $\sqrt{P}$, while Barnes–Hut can raise it almost linearly (subject to its logarithmic factor). A fair benchmark must report accuracy criterion, timestep, particle distribution, force cutoff/opening criterion, and both serial and parallel reference execution methods.

Conclusion: the useful-compute scaling law determines $N(P)$; it does **not** uniquely determine $\alpha(P)$ without a model for setup, communication, and synchronization. Gustafson's law works best when these terms remain a small fraction of the measured runtime.

Task 19 — Monte Carlo: Near-Ideal Weak Scaling with Nonzero Overhead

Ideal embarrassingly parallel case

Suppose P workers each draw k independent samples and evaluate a fixed-cost integrand. Useful serial-equivalent work scales as $W(P) \propto kP$, while per-worker computation takes approximately $\tau(k)$ seconds. If there is no cross-worker interaction and no fixed setup, then $T_P \approx \tau$, $T_1 \approx P\tau$, $S(P) \approx P$, and $E(P) \approx 1$.

Real Monte Carlo systems still contain overhead, even when sample evaluations are mutually independent:

- **PRNG seed/stream management:** central generation or validation, stream partitioning, or accidental serial initialization. Correct independent streams are important for statistical validity; assigning each worker an arbitrary unvalidated seed is not a guarantee of independence.
- **Reduction operations:** combining partial sums, variances, or histograms and synchronizing workers.
- **Output/checkpoint serialization:** logging, writing one file, serial aggregation, metadata updates, and reproducibility bookkeeping.
- **Load imbalance:** rejection sampling, adaptive paths, variable path lengths, and heterogeneous execution rates.

Quantify tree-reduction latency and its effect on speedup

Let a be constant serial setup/output seconds and b be nonoverlapped seconds per reduction-tree level. A balanced binary reduction has $\lceil \log_2 P \rceil$ stages, so a simplified runtime is

$$T_P(kP) = \tau(k) + a + b\lceil \log_2 P \rceil.$$

If serial execution of all kP samples takes $P\tau(k)$ and incurs no equivalent network reduction, the useful scaled speedup is

$$\boxed{S_{MC}(P) = \frac{P\tau(k)}{\tau(k) + a + b\lceil \log_2 P \rceil}}.$$

The lost factor relative to ideal is $S_{MC}/P = \tau/[\tau + a + b\lceil \log_2 P \rceil]$. Thus logarithmic overhead becomes significant when useful compute per worker is small. A naive sequential reduction that sends P messages to one root could cost $O(Pb)$ stages instead of $O(b \log P)$, a potentially catastrophic difference.

Worked numerical scenario

For illustration, let $P = 1024$, $\tau = 1.0$ s per worker, $a = .001$ s, and $b = .0001$ s per level. Since $\log_2(1024) = 10$,

$$T_P = 1.000 + .001 + .001 = 1.002 \text{ s},$$

$$S_{MC} = 1024/1.002 \approx \boxed{1021.9561}, \qquad E \approx 0.99800399.$$

The observed setup-plus-reduction runtime fraction is $\gamma = .002/1.002 \approx 0.0019960$. The generalized overhead-aware formulation from Task 12 gives $S \approx P(1-\gamma)$ when all useful computation is parallel and the serial equivalent excludes the network overhead; this reproduces the example. If the same engine were run with $\tau = .01$ s, its overhead would be 20% of the useful compute and efficiency would fall to $0.01/(.012) \approx 83.33\%$ despite an unchanged tree.

Statistical correctness is not implied by speedup

Monte Carlo speedup measurements must preserve sampling distribution, counter-based streams or jump-ahead guarantees, deterministic reduction order if bitwise repeatability is desired, and uncertainty of the final estimator. Parallel reduction changes floating-point summation order, so exact bitwise equality can fail even when the statistical estimator is valid. Compute speedup without checking statistical correctness is incomplete.

Task 20 — Why BFS and PageRank Resist Regular Weak Scaling

Weak-scaling promise vs. the graph execution reality

For an ideal dense regular kernel, work is distributed in equal blocks, arithmetic intensity is high, and communication patterns are predictable. Increasing global problem size with processor count can preserve **nearly identical local work and communication**. A power-law graph does not have these properties merely because the number of edges grows linearly with the number of processors.

Breadth-First Search (BFS) processes a sequence of frontiers. Work per level depends on visited vertices and frontier edges, which can vary by orders of magnitude. A few high-degree hubs account for disproportionate neighbor exploration. Fixed average edges per worker does not imply fixed **maximum** edges per worker, and the slowest partition determines completion of each barrier.

PageRank iteratively gathers and distributes contributions across irregular adjacency lists. Even if PageRank has a more regular iteration count than BFS, heavy-tailed in-degrees create hot destinations, skewed atomics, and memory bandwidth contention. Distributed graph partitions may need many remote messages and global convergence tests.

Four scaling bottlenecks

Source	BFS manifestation	PageRank manifestation	Effect on weak scaling
Irregular memory access	Frontier-dependent neighbor scans	Sparse gather/scatter	Cache/TLB misses, lower useful FLOP/s
Partition imbalance	Hubs and uneven frontiers	Unequal edge counts and high-in-degree vertices	Tail latency sets level/iteration time
Remote communication	Cross-partition frontier notifications	Contributions to remote vertex owners	Network startup, congestion, all-to-all-like traffic
Synchronization	Barrier at each BFS level	Residual/convergence reductions per iteration	Growing global collective costs

A simplified performance inequality

Let $E(P)$ be edges and assume $E(P) = eP$, so aggregate edge count grows linearly. Even with ideal average throughput r edges per second per worker, the implementation may have

$$T_P \gtrsim \max_i \left(\frac{E_i(P)}{r} \right) + C(P) + I(P),$$

where $E_i(P)$ is the edge count actually visited by partition i (not just average assigned edges), $C(P)$ communication and barriers, and $I(P)$ additional congestion/imbalance. Linear growth of $\sum_i E_i$ is insufficient to bound $\max_i E_i$ or $C(P)$. For BFS, varying graph diameter and frontier count add another serial critical-path factor. For PageRank, even a fixed iteration count can be dominated by random memory access and cache misses.

Practical countermeasures and evaluations

Use edge-balanced/2-D partitioning, hotspot replication, push/pull BFS switching, dynamic frontier scheduling, compressed sparse layout, communication aggregation, and asynchronous or relaxed-accuracy convergence when scientifically acceptable. Each trade-off has its own cost: replication uses memory, asynchronous updates can change convergence, and repartitioning itself consumes time.

A robust study should compare at least (i) vertex count, edge count, graph degree distribution, and diameter; (ii) maximal and average edges handled by a worker; (iii) communication bytes and global synchronizations; (iv) total elapsed time and efficiency; and (v) output correctness. Merely plotting a weak-scaled vertex count against P can conceal severe work skew.

Contrast with dense GEMM (Task 16): GEMM partitions uniform tiles and uses highly optimized contiguous BLAS kernels, while graph analytics operate on data-dependent sparse neighborhoods. Thus even when both have increasing workloads, the **critical path, data movement, and work variance** are fundamentally different.

Part V — Benchmarking, Empirical Research & Repository Artifacts

Task 21 — Parametric Speedup Simulator Implementation

Deliverables

The executable Python program `benchmarks/simulator.py` generates two forms of output:

1. `benchmarks/simulated_scaling.csv`, with processor count, serial fraction, Amdahl speedup, Gustafson scaled speedup, and Gustafson parallel efficiency.
2. `plots/simulator_overlay.png` and `plots/simulator_overlay.pdf`, overlaying ideal, strong-scaling, and weak-scaling curves.

The command line accepts arbitrary comma-separated integer processor counts $1 \le P \le 65{,}536$ and floating serial fractions $0.001 \le \alpha \le 0.20$. It rejects out-of-range and malformed inputs instead of silently clipping them. This makes the simulator suitable for both a few hand-picked configurations and the maximum assigned range.

Equations encoded

$$S_A(P;s) = \frac{1}{s + (1-s)/P}, \qquad S_G(P;\alpha) = P - \alpha(P-1), \qquad E_G(P;\alpha) = S_G(P;\alpha)/P.$$

The same **numeric** fraction is used as s and α in the comparison plot; as established in Task 08, these symbols belong to different workload frames. Plots therefore compare model curves rather than claiming they predict the same physical benchmark.

Reproduce the default data and figure

```
python3 benchmarks/simulator.py
```

Try an arbitrary valid subset:

```
python3 benchmarks/simulator.py \
  --core-counts 1,3,17,256,65536 \
  --alphas 0.001,0.075,0.20 \
  --csv benchmarks/custom_simulated_scaling.csv \
  --plot plots/custom_overlay.png
```

The output CSV contains $17 \times 4 = 68$ rows for the shipped default sweep (17 different P values and 4 fractions). The program writes both PNG and vector PDF plot for the same run. Plots use a log2 processor axis and log10 speedup axis to display the full range up to 65,536 without squeezing the small-P behavior.

Verification criteria

- At $P = 1$, both models equal 1 for all valid fractions.
- For $\alpha = 0.001$ and $P = 65,536$, Gustafson predicts $65,470.465$; Amdahl predicts $1/(.001 + .999/65536) \approx 984.985$.
- All result rows satisfy $1 \le S_G \le P$, $0 < E_G \le 1$, and $1 \le S_A \le P$ in the parameter range.
- Automated formula boundary tests are included in `tests/test_formulas.py`.

Quality note: the simulator implements **ideal formulas**, not empirical capacity or network constraints. Benchmarks Tasks 22–24 analyze real deviations.

Task 22 — OpenMP Weak-Scaling Benchmark Suite

Design

The benchmark computes a deterministic numerical quadrature with **constant work per thread** `k`. At thread count `P`, the measured job contains `P*k` samples. The C++ executable measures both `T1(P*k)`

(a single thread processes all P partitions) and `TP(P*k)` (an OpenMP team of P threads processes the same P partitions). These times share exactly the same scaled workload, so `S_scaled = T1(P*k) / TP(P*k)` is a valid scaled-speedup estimate.

The checksum verifies equivalence to relative error below `1e-10`. Threads perform independent math and synchronize only for the OpenMP reduction. The benchmark excludes process launch from timed sections but **includes** team creation, join, and reduction in the parallel timed section. Per-config timings cover compute only, not file I/O.

Run

```
cd gustafson-law-practice
python3 benchmarks/weak_scale_bench/run_benchmarks.py --runs 5 --work-per-thread 2000000
python3 benchmarks/fit_gustafson.py
python3 benchmarks/plot_comparison.py
```

Requirements: Python 3, `g++` with OpenMP support, and the Python libraries in `requirements.txt`. `run_benchmarks.py` compiles `bench.cpp` into an untracked local executable. `--threads 1,2,3,4,5` specifies an explicit sweep, constrained by the host's available processor affinity; `auto` (default) includes 1 and the maximum CPU affinity count and uses all intermediate counts when at most 16 are available. The RNG seed controls **only execution order**, not the deterministic compute kernel. The script shuffles configuration order within each repetition to reduce sequential thermal bias.

The total dataset is `number_of_thread_counts × number_of_runs`; this submission includes five independent repetitions at each of five thread counts. Results are one experimental snapshot, not universal hardware performance claims. In a VM or container the CPU quota, competing tenants, and unseen actual memory channels can alter speedup.

Task 23 — Experimental Measurements and Least-Squares Fit

Experiment actually executed

The submitted `benchmarks/results.csv` contains **25 raw trials**: **five independent executions** for each of the **five distinct OpenMP thread counts** $P \in \{1, 2, 3, 4, 5\}$. Each worker performs $k = 12{,}000{,}000$ deterministic quadrature samples; total samples for a run scale as $W(P) = kP$. Each trial independently measures

$$T_1(kP) \quad \text{and} \quad T_P(kP),$$

using the **same total workload**. Therefore the empirical scaled speedup is $S_{\text{obs}}(P) = T_1(kP)/T_P(kP)$, not the strong-scaling speedup of one constant-sized job. Every trial includes two high-resolution timings, a numerical checksum comparison, and the run ID. The trial ordering was shuffled inside each repetition; all 25 raw observations, including slow outliers, are retained.

Sample statistics (from the actual run)

Threads	Trials	Mean serial ref. $T_1(kP)$, s	Mean parallel $T_P(kP)$, s	Mean observed S	Sample std. of S	Std. error of S	Mean efficiency
1	5	0.06991	0.06699	1.04957	0.18391	0.08225	1.04957
2	5	0.14239	0.08054	1.77157	0.23324	0.10431	0.88579
3	5	0.24355	0.13430	1.97922	0.95621	0.42763	0.65974
4	5	0.26848	0.16034	1.68781	0.19240	0.08605	0.42195
5	5	0.38780	0.33772	1.82375	0.76730	0.34315	0.36475

For each processor count, sample standard deviation uses divisor $n-1$ and standard error uses $s/\sqrt{n}$ with $n = 5$. The reported *mean of per-trial speedup ratios* is not generally equal to the ratio of aggregate

mean times. Small deviations such as the $P = 1$ mean speedup of 1.05 are measurement noise, not a claim that one worker intrinsically exceeds one worker.

Explicit least-squares formulation

Gustafson's ideal model is $S(P) = P - \alpha(P - 1)$. Rewrite as

$$y_i = P_i - S_{\text{obs},i}, \qquad x_i = P_i - 1, \qquad y_i \approx \alpha x_i.$$

The **unweighted OLS** coefficient through the theoretical origin, using **all raw trial rows with** $P > 1$, is

$$\boxed{\hat{\alpha} = \frac{\sum_i x_i y_i}{\sum_i x_i^2} = 0.7303855508}.$$

Its physically constrained counterpart $\min(1, \max(0, \hat{\alpha}))$ is also **0.7303855508**. A stratified nonparametric bootstrap that resamples the five trials separately within each processor count (2,000 replicates, fixed seed) gives an indicative 95% interval approximately **[0.63898, 0.83589]** for the raw coefficient.

Model-fit diagnostics and interpretation

The fitted unweighted speedup RMSE is **0.6050**, and the conventional speedup-space R^2 is **-0.0072**. A negative R^2 signals that a single constant-α line fits these noisy measurements poorly. The effective fitted α is **not** evidence that 73% of the numerical quadrature algorithm is inherently sequential. The kernel is largely data parallel; the fit absorbs runtime overhead, VM scheduling, core-quota limits, and benchmark-reference noise into one poor-fit phenomenological parameter.

The fifth trial at $P = 5$ has a parallel time about **1.019 s**, versus roughly 0.150–0.183 s for the other four $P = 5$ trials; it is kept because discarding it without a predefined exclusion rule would bias the study. Host affinity advertises five CPUs but the cgroup has a four-core CPU quota; these are not five unrestricted physical cores. Shared-system interference and unobserved power management are possible contributors, not confirmed causes.

Limitations: five trials per count are enough for the assignment but insufficient to characterize noisy virtual infrastructure robustly. The bootstrap interval conditions on this small trial collection; it does not capture hidden system-level biases or prove a constant causal serial fraction. Re-run the harness on a dedicated, frequency-controlled host for stronger inference.

Reproduce:

```
python3 benchmarks/weak_scale_bench/run_benchmarks.py --runs 5 --work-per-thread 12000000
python3 benchmarks/fit_gustafson.py
```

Task 24 — Publication Plots and Hardware Audit

Reproducible figures

The script `benchmarks/plot_comparison.py` reads `benchmarks/aggregate.csv` and `benchmarks/fit_summary.json` to produce:

- `plots/scaling_comparison.png` — 260-dpi raster, and matching **vector** `plots/scaling_comparison.pdf` and `.svg`.
- `plots/efficiency_comparison.png` and vector PDF — empirical efficiency and fitted efficiency.

The main speedup plot contains all four requested traces: **(1)** ideal $S = P$, **(2)** Gustafson $S = P - \hat{\alpha}(P - 1)$, **(3)** empirical measured mean S with **standard-error bars**, **(4)** Amdahl $S = 1/[\hat{\alpha} + (1 - \hat{\alpha})/P]$. Using the same numerical coefficient in both laws is illustrative; their serial shares have different definitions.

Actual system audited

Property	Observation	Confidence or limitation
CPU model	**Intel(R) Xeon(R) Platinum 8573C**	Visible vCPU model exposed by Linux `/proc/cpuinfo`
Visible logical processors	**5**, affinity CPUs `0`–`4`	Five runnable OS-visible CPUs, not proof of five dedicated physical cores
Cgroup quota	**400000** / **100000** microseconds = **4.0 CPU cores**	Important limit; five active threads can exceed the combined CPU allowance
OS / architecture	Linux 6.18.57, x86-64, virtualized environment	Host scheduling is not under experimental control
L1 data cache	**48 KiB** for the CPU0 cache entry, shared list `0`–`1`	Observed sysfs `cpu0/cache/index0`, not an assertion of physical device topology
L1 instruction cache	**32 KiB**, CPU0 entry	Observed sysfs cache entry
L2 unified cache	**2 MiB**, CPU0 entry	Observed sysfs cache entry
L3 unified cache	**260 MiB**, visible shared list `0`–`4`	Virtualized visible topology; actual host placement may differ
NUMA	**1 NUMA node visible (node 0)**	No NUMA crossing can be demonstrated on this exposure
Memory channels	**Not disclosed to unprivileged environment**	**Unknown**, no invented physical-channel count
Temperature and clock throttling	**No direct telemetry collected**	Cannot attribute a slowdown to throttling with confidence
Compiler	**g++ 14.2.0**, OpenMP with `-O3`	Recorded audit and reproducible build flags

Detailed machine-readable evidence, including the full `lscpu` output, CPU affinity, cache index attributes, cgroup quota, and audit UTC timestamp, is in `analysis/hardware_audit.json`. Regenerate it with `python3 benchmarks/audit_hardware.py`.

Explain the observed departure from ideal scaling

The empirical speedups in Task 23 fail to follow P or a clean constant-α model. **Directly supported facts** are: the four-core CPU quota despite five visible CPU IDs; substantial between-trial variance; a slow $P = 5$ run; and a poor R^2 for the constant-α regression. This is consistent with CPU-share contention, virtualization jitter, affinity/scheduling overhead, and OpenMP synchronization.

Not established: thermal throttling (no validated clock/temperature time series), memory-channel contention (channel count not available), and NUMA-boundary crossing (only one node exposed). These are plausible general causes of parallel-performance deviation, but they should **not** be asserted as measured causes for this dataset. The numerical kernel is compute-oriented with small memory demand, so its results do not generalize to cache- or bandwidth-intensive production codes.

Verification and rerun

```
python3 benchmarks/audit_hardware.py
python3 benchmarks/fit_gustafson.py
python3 benchmarks/plot_comparison.py
```

To obtain publication-grade evidence rather than just publication-quality **figures**, repeat on a reserved machine, record per-core effective frequency and throttling counters, measure memory bandwidth with an appropriate tool, pin OpenMP threads, predeclare any exclusion rule, and extend the number of repetitions. The submitted figures correctly describe **this measured container snapshot**.

Task 25 — Master Repository and Executive Synthesis

Course: Parallel & Distributed Computing Architecture
Assignment: 25-task comprehensive practice sheet (instructor: Sufyan bin Uzayr)
Submission target: individual Git repository, branch or folder `gustafson-law-practice/`
Status: all 25 tasks answered; executable simulation, OpenMP benchmarking, real measured data, regression analysis, validation tests, and publication-resolution plots included.

> **Attribution:** The assignment prompt is included in `references/Gustafson_Law_Practice_Tasks.pdf`. This directory contains an independent worked solution. The repository has a **local Git history** but has **not** been pushed to the student's personal Git hosting account. Review and personalize attribution before submitting.

Table of contents

1. Executive synthesis
2. Essential mathematics
3. Contents and task-by-task index
4. Reproduce the scripts and benchmarks
5. Actual experiment and its limits
6. When to use each model
7. Primary references
8. Submission checklist

Executive synthesis

Amdahl's Law models the speedup of the **same fixed problem** on more processors; when the original one-core serial work fraction is $s_A > 0$, total speedup is bounded by $1/s_A$. **Gustafson's Law** models how much **more useful work** can be completed in nearly the **same wall-clock time**, with α measured on the *parallel scaled* job. In its ideal form, $S_G(P) = P - \alpha(P-1)$ and $E_G(P) = 1 - \alpha + \alpha/P$. Neither invalidates the other: they answer distinct questions.

The Sun–Ni model incorporates **aggregate memory capacity**, which may permit workload growth faster or slower than the fixed-time rule. Isoefficiency analysis makes the missing constraint explicit: constant local work preserves constant efficiency only when communication, imbalance, and coordination overhead do not outgrow useful work. Dense GEMM, regular stencils, N-body solvers, Monte Carlo, and graph workloads demonstrate the diverse behavior of realistic scaling strategies.

A complete engineering scaling report must always name **(i)** the problem-size schedule $W(P)$, **(ii)** the numerator of speedup (serial reference time for *the same job*), **(iii)** the measured serial fraction's frame, **(iv)** computation vs communication, and **(v)** memory and hardware constraints. A large theoretical scaled speedup is an upper-bound model, not a hardware guarantee.

Essential mathematics

Model	Formula	Experiment
Amdahl fixed size	$S_A(P) = 1/[s_A + (1-s_A)/P]$	$W(P) = W_0$, s_A measured on one core
Gustafson scaled	$S_G(P) = P - \alpha(P-1)$	$T_P(W(P)) \approx T^*$, α measured on the scaled parallel run
Gustafson efficiency	$E_G(P) = 1 - \alpha + \alpha/P$	Constant α idealization yields limiting efficiency $1-\alpha$
Communication-aware	$S = (A+B)/(A+B/P+C)$	Same scaled useful work $A+B$; C is nonoverlapped parallel communication
Isoefficiency	$E = W/[W + T_o(P,W)]$	Keep W large enough relative to parallel overhead
Memory-bounded	$S_M = (A+h(P)B)/(A+h(P)B/P)$	Scale useful work with memory via $h(P) = g(Pm)/g(m)$

Contents and task-by-task index

All required files are included **at their specified paths**, with additional reproducibility artifacts where helpful.

Task	Topic	Answer / submitted deliverable
01	Strong vs. weak foundations	`theory/task01_scaling_models.md`
02	Scaled speedup derivation	`theory/task02_derivation.md`
03	Serial fraction reference frame	`theory/task03_serial_invariance.m`
04	Asymptotic limits	`theory/task04_asymptotic_limits.m`
05	Historical 1024-node nCUBE	`theory/task05_historical_ncube.md`
06	Parametric table $\alpha = .04$	`calculations/task06_parametric_ta`
07	Maximum α for $P = 8192$	`calculations/task07_tolerable_ser`
08	Amdahl vs Gustafson comparison	`calculations/task08_cross_diverge`
09	Multi-phase workload	`calculations/task09_multicomponen`
10	3-D grid memory ceilings	`calculations/task10_memory_growth`
11	Sun–Ni memory-bounded scaling	`theory/task11_sun_ni_comparison.m`
12	Communication-penalty model	`theory/task12_communication_penal`
13	Counterexamples	`theory/task13_failure_cases.md`
14	Isoefficiency proof	`theory/task14_isoefficiency.md`
15	Energy & power scaling	`theory/task15_energy_scaling.md`
16	Dense GEMM dimension scaling	`calculations/task16_gemm_scaling.`
17	3-D stencil communication	`calculations/task17_stencil_scali`
18	Direct vs Barnes–Hut N-body	`analysis/task18_nbody_scaling.md`
19	Monte Carlo reduction	`analysis/task19_monte_carlo.md`
20	BFS/PageRank graph scaling	`analysis/task20_graph_processing.`
21	Python parametric simulator	`benchmarks/simulator.py`; `simulated_scaling.csv`; `analysis/task21_simulator_validation.m`
22	Real OpenMP benchmark harness	`benchmarks/weak_scale_bench/bench` `run_benchmarks.py`; run instructions
23	25 raw trials and regression	`benchmarks/results.csv`; `fit_gustafson.py`; `aggregate.csv`; `fit_summary.json`; analysis
24	Publication-quality plots and audit	`plots/scaling_comparison.pdf` / PNG / SVG; `analysis/task24_hardware_audit.md`; `hardware_audit.json`; `plot_comparison.py`
25	Executive master documentation	**This `README.md`**

Additional diagrams: `plots/simulator_overlay.png/.pdf` and `plots/efficiency_comparison.png/.pdf`. Automated checks are in `tests/test_formulas.py`. Primary sources are cited inside relevant solutions; all reported experiment figures are locally produced, not fabricated.

Reproduce the scripts and benchmarks

Tested with Python 3, g++ 14 with OpenMP, NumPy and Matplotlib. From the **root directory**:

```
python3 -m pip install -r requirements.txt

# Task 21 — generate ideal model data and overlay figures
python3 benchmarks/simulator.py

# Task 22 — compile OpenMP and execute 5 independent runs at P=1..5
python3 benchmarks/weak_scale_bench/run_benchmarks.py \
  --threads 1,2,3,4,5 --runs 5 --work-per-thread 12000000

# Task 23 — aggregate, calculate standard errors, fit alpha by OLS
python3 benchmarks/fit_gustafson.py

# Task 24 — audit CPU/caches/quota and create raster/vector figures
python3 benchmarks/audit_hardware.py
python3 benchmarks/plot_comparison.py

# Verify numerical identities, dataset shape and exported figures
python3 -m unittest discover -s tests -v
```

Hardware adaptation: choose a subset of available thread counts if your environment has fewer than five affinity-visible processors. To reproduce the exact submitted 25-row format, use five counts and five runs; when fewer than five are visible on your device, execute in an appropriately provisioned environment. Without an explicit thread list, the script automatically samples from one CPU to its local affinity maximum. Do not add artificial trials to satisfy the count.

Rebuild the consolidated PDF The optional PDF builder requires Pandoc and a LaTeX installation with XeLaTeX and the **Noto Serif**, **Inter**, and **DejaVu Sans Mono** fonts; the executable calculations themselves do not require LaTeX. From the root directory:

```
python3 tools/build_report.py
```

This collects the full Task 01–25 Markdown solutions into `Complete_Solutions.pdf`, with a navigable table of contents, exact equations, readable tables, and measured plots. Any change to the Markdown answers should be followed by rebuilding this PDF.

The OpenMP benchmark measures scaled speedup correctly: the **single-core reference works on the same scaled Pk samples** as the P-core run. It checks numerical equivalence in each execution and records raw timings rather than back-calculating them from ideal models. The compiled binary is intentionally untracked and rebuilt by the harness.

Actual experiment and its limits

This project was measured in a Linux container with **five visible CPU IDs but a quota corresponding to four CPU cores**. There are five independent trials for each of five configurations; the empirical speedup data is noisy and includes an extreme $P = 5$ outlier. The constant-α least-squares fit returns $\hat{\alpha} \approx 0.7304$ with a very poor model fit ($R^2 \approx -0.0072$). That coefficient should **not** be interpreted as a causal property of the mathematical kernel. No reliable thermal-throttle telemetry, physical memory-channel count, or cross-NUMA experiment was available. Read Task 23 and Task 24 before comparing the figure with production cluster data.

When to use each model

- **Choose Amdahl/strong scaling** for fixed-demand service latency, a fixed mesh, a fixed query, or a fixed input file when additional cores do *not* increase the scientific task.
- **Choose Gustafson/weak scaling** for simulations with meaningful higher resolution, independent Monte Carlo paths, or larger regular batch-processing jobs when constant local work and sufficient memory/network resources are plausible.
- **Choose Sun–Ni memory-bounded scaling** when a larger problem becomes possible specifically because more aggregate memory is available, even if it requires a longer runtime.
- **Use isoefficiency + profiling** when synchronization, communication, load imbalance, algorithmic work inflation, or memory capacity threaten the ideal scaling schedule.

Engineering rule: report both **scaled throughput** and **time-to-solution** where possible; include parallel efficiency, energy per result, and a clear description of scientific accuracy and dataset characteristics. Processor count alone never identifies quality of scaling.

Primary references

1. Gene M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS (1967), 483–485.
2. John L. Gustafson, "Reevaluating Amdahl's Law," *Communications of the ACM* 31(5), 532–533 (1988).
3. John L. Gustafson, Gary R. Montry, Robert E. Benner, "Development of Parallel Methods for a 1024-Processor Hypercube," *SIAM J. Sci. Stat. Comput.* 9(4), 609–638 (1988).
4. Xian-He Sun and Lionel M. Ni, "Scalable Problems and Memory-Bounded Speedup," *J. Parallel Distrib. Comput.* 19(1), 27–37 (1993).

Submission checklist

- ☒ All 25 tasks represented by solution files or executable code, with derivations and arithmetic shown.
- ☒ Git-ready repository organized into `theory/`, `calculations/`, `benchmarks/`, `analysis/`, and `plots/`.
- ☒ Parametric simulator with bounds, CSV, and overlay plot.
- ☒ OpenMP benchmark, five thread counts, five independent real runs per count, and numerical correctness checks.
- ☒ Least-squares fit, raw and aggregate CSV, standard errors, and fitted/ideal/Amdahl figures.
- ☒ Hardware audit with truthful unknowns for unobservable telemetry.
- ☒ Eight automated checks passing in this environment.
- ☒ Local Git commits prepared for repository import.
- ☐ **Student action:** personalize repository ownership and `git remote`, review the analysis, and push to the designated personal Git repository.

Academic integrity / reproducibility: if rerunning on another machine, the measured values and hardware audit will differ. Keep an unmodified copy of each actual raw run and regenerate Task 23/24 reports to match your environment. Do not submit these container measurements as if you personally collected them on a different computer.

Appendix — Verified Benchmark Figures

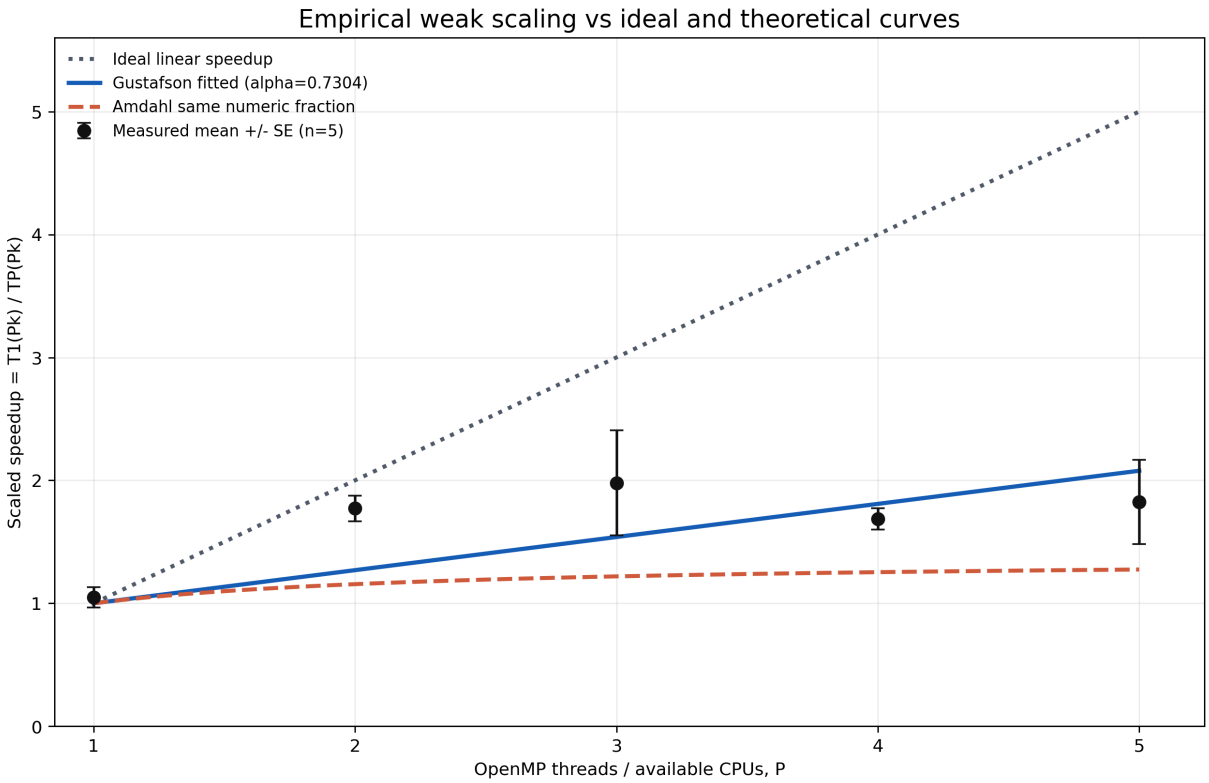


Figure 1: Observed mean scaled speedups, error bars, and model curves

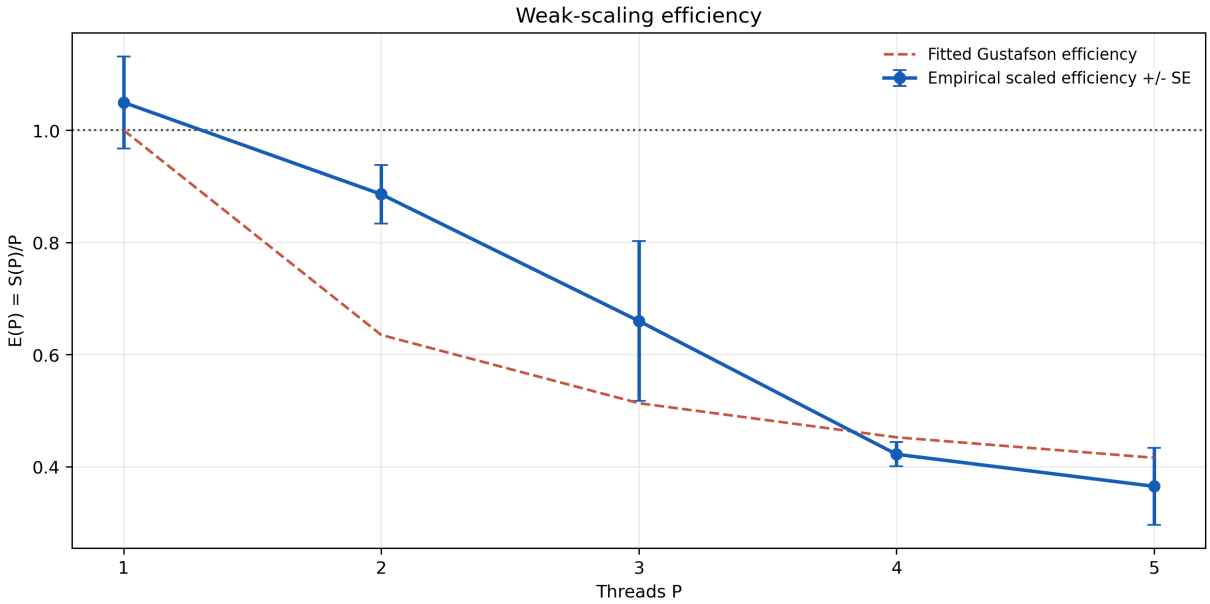


Figure 2: Measured parallel efficiency and fitted efficiency

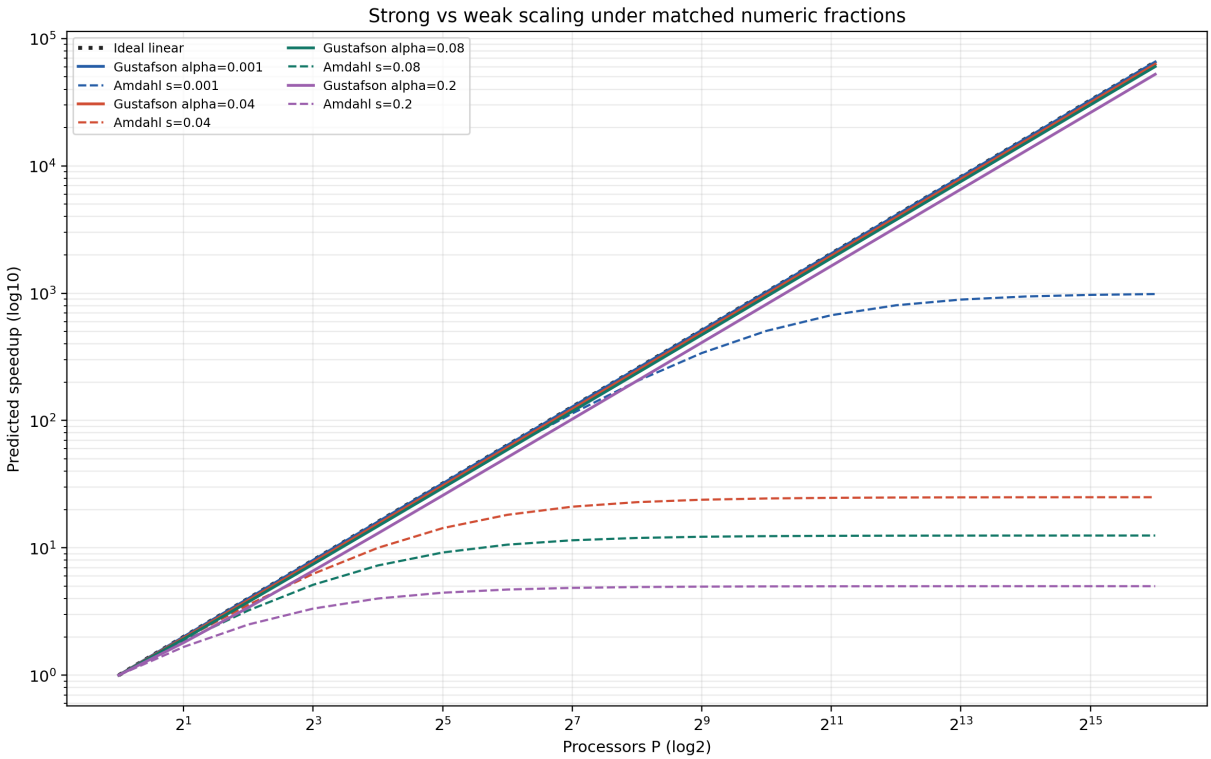


Figure 3: Parametric simulator's fixed-size and fixed-time overlays